

Life Sequences as Language: A Complete Guide to the `pop2vec` LLM Pipeline

BERT-style (MLM) and GPT-style (AR) transformers over Dutch registry life-event data

Documentation generated from the codebase
repository: `life-sequencing-dutch`, module: `pop2vec/llm`

July 5, 2026

Abstract

This report explains, end to end, how the `pop2vec` codebase turns Dutch administrative registry data into *life sequences*, pretrains a transformer on them (extending *life2vec*, arXiv:2306.03009), and produces downstream predictions in two ways: (1) **static** — freeze the pretrained encoder, export one embedding per person, and train a small MLP on those embeddings; and (2) **fine-tuned** — attach a task head to the pretrained encoder and train the whole network per task. The same encoder supports two pretraining objectives selected by the `training_task` hyperparameter: `mlm` (BERT-style masked language modelling, the default) and `ar_lm` (GPT-style autoregressive next-token prediction with causal attention). Every section includes diagrams in the spirit of the BERT and “Attention is All You Need” papers, and maps each concept to the exact file and function that implements it.

Contents

1	The Big Picture	3
2	Data: From Registry Tables to Token Sequences	3
2.1	Attribute preprocessing (<code>preprocess_data.py</code>)	3
2.2	Sequence construction and tokenization (<code>pipeline.py</code> + <code>tasks/mlm.py</code>)	4
3	Model Architecture	5
3.1	Embedding block: how a token meets its time	5
3.2	Attention: Performer with local heads	5
3.3	Heads	6
3.4	Reference hyperparameters	6
4	Step 1 — Pretraining (<code>pretrain.py</code>)	6
4.1	BERT mode: masked LM + sequence-order CLS task	7
4.2	GPT mode: autoregressive next-token prediction	7
4.3	Optimization and infrastructure (both modes)	7
5	Step 2a — Static Route: Embedding Inference (<code>infer_embedding.py</code>)	8
6	Step 3a — Static Route: MLP Prediction (<code>train_simple.py</code>)	8
7	Step 2b — Finetuned Route (<code>finetune_runner.py</code> + <code>finetune_model.py</code>)	9
7.1	Training recipe (per target column)	9
8	Static vs. Finetuned, Side by Side	10

9 File Map and Config Cheat-Sheet 10
9.1 Who does what 10
9.2 Key config fields per step 11
10 Frequently Confusing Points (Read This Twice) 11

1 The Big Picture

The core idea inherited from life2vec: **treat a person’s life as a document**. Every life event (a job change, a move, a hospital visit, a marriage) becomes a *sentence*; every attribute of the event becomes a *token* (e.g. `income_percentile_87`). A transformer is pretrained on millions of such documents, exactly like BERT/GPT on text, and the learned representations are then used to predict life outcomes.

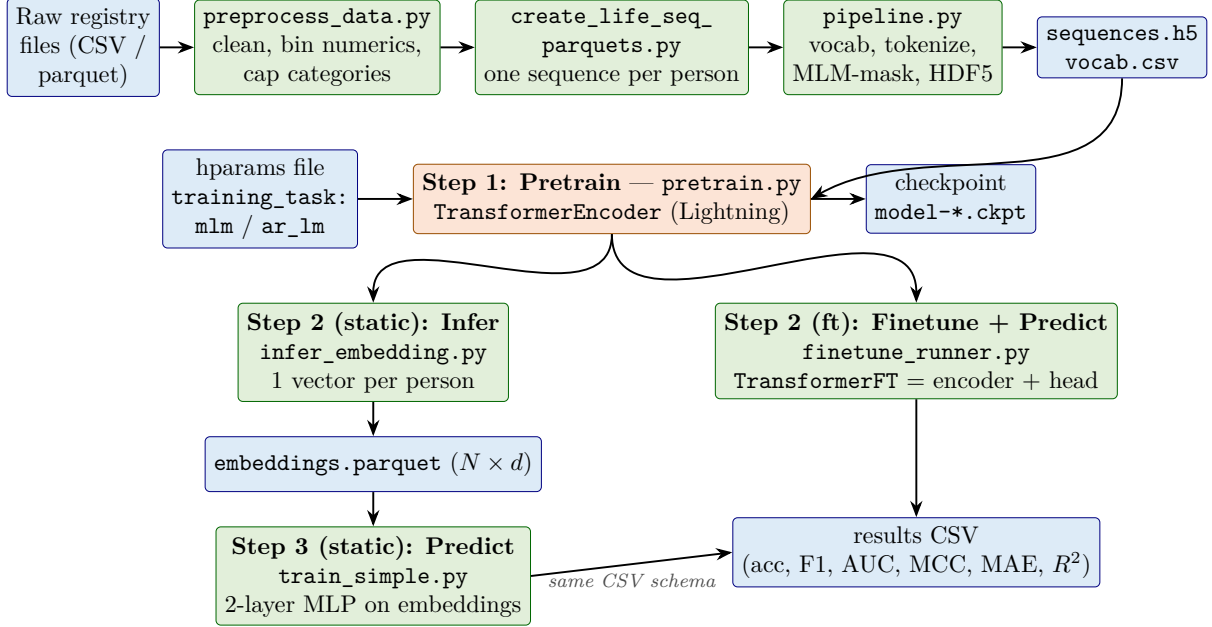


Figure 1: The full pipeline. The two downstream routes (*static* left, *finetuned* right) both start from the same pretrained checkpoint and report results in an identical CSV schema, so numbers are directly comparable.

Your workflow, confirmed. The four entry points you listed are exactly right:

1. **Pretrain:** `pop2vec/llm/src/new_code/pretrain.py`
- 2a. **Static, infer:** `pop2vec/llm/src/new_code/infer_embedding.py`
- 2b. **Finetune + predict:** `pop2vec/llm/src/new_code/finetune_runner.py`
- 3a. **Static, predict:** `pop2vec/llm/src/new_code/train_simple.py`

The only step upstream of these is data preparation (Section 2), which produces the tokenized HDF5 file and the vocabulary CSV that `pretrain.py` consumes.

2 Data: From Registry Tables to Token Sequences

2.1 Attribute preprocessing (`preprocess_data.py`)

Each raw table (CSV or parquet, one row per event per person, keyed by `RINPERSON`) is normalized so that every column becomes a small categorical alphabet:

- **Numeric columns** are converted to **percentiles** (`transform_to_percentiles`), so “income = 43,210 EUR” becomes “income is in percentile bucket 87”. This is what makes a continuous value usable as a token.
- **Categorical columns** keep only the **top 100** most frequent values; the rest are renamed `0thers` (`replace_less_frequent`).
- Missing values become the literal token `MISSING`; rows without the two time columns (`age`, `days_since_first`) are dropped; columns matching restricted substrings are removed.

The result: every event attribute has at most ~ 100 distinct values, keeping the final vocabulary compact.

2.2 Sequence construction and tokenization (pipeline.py + tasks/mlm.py)

For each person, events are ordered in time and encoded by `MLM.encode_document()`:

1. A **prefix sentence** is built: `[CLS]` + background tokens (fixed characteristics such as gender, birth year, origin) + `[SEP]`.
2. Every **event** becomes a sentence of tokens (one token per attribute, spelled `attribute_value`) terminated by `[SEP]`.
3. If the whole document exceeds `max_length` (512 in the standard config), the *oldest* events are dropped — the code keeps the most *recent* events that fit (suffix truncation).
4. People with fewer than 5 events (`MIN_EVENT_THRESHOLD`) are skipped.

Each sequence is stored as a $4 \times L_{\max}$ integer matrix (`input_ids`), i.e. **four parallel streams** per token position:

	background prefix				event 1			event 2		
token	[CLS]	gender_M	birth_1975	[SEP]	job_teacher	income_p54	[SEP]	moved_Utrecht	[SEP]	[PAD]
abspos	0	0	0	0	3650	3650	3650	5113	5113	0
age	0	0	0	0	25	25	25	29	29	0
segment	0	0	0	0	1	1	1	2	2	0
pad. mask	1	1	1	1	1	1	1	1	1	0

Figure 2: One person’s encoded life sequence (toy example). `input_ids` has shape $(4, L_{\max})$: token ids, absolute position in days (`abspos`), age at the event, and segment id. All tokens of the same event share the same `abspos`/`age`/`segment`. The prefix carries zeros for all three time streams. `padding_mask` marks real tokens.

The encoded corpus is written to HDF5 with these datasets:

dataset	shape	meaning
<code>input_ids</code>	$(N, 4, L_{\max})$	the 4-stream input above (token stream masked if MLM-encoded)
<code>padding_mask</code>	$(N, L_{\max})$	1 = real token, 0 = padding
<code>original_sequence</code>	$(N, L_{\max})$	unmasked token ids (MLM files only)
<code>target_tokens</code>	(N, T)	true ids of masked positions, -1 -padded (MLM only)
<code>target_pos</code>	(N, T)	positions that were masked (MLM only)
<code>target_cls</code>	$(N,)$	sequence-order label 0/1/2 (MLM only, Sec. 4.1)
<code>sequence_id</code>	$(N,)$	RINPERSOON of the person

Vocabulary. `CustomVocabulary` scans all data files and writes `vocab.csv` with columns `TOKEN`, `CATEGORY`, `ID`. IDs 0–8 are reserved general tokens; the important ones are `[PAD]` = 0, plus `[CLS]`, `[SEP]`, `[UNK]`, `[MASK]`, and (in real data) a death token `[death]_death`. `pretrain.py` reads `vocab_size` as simply the number of rows of this CSV and looks up the special ids with `load_special_ids()` (`utils.py`).

Data loading. `load_data.py` provides several dataset classes; the one used by pretraining and inference is `CustomLazyHDF5Dataset`: it opens the HDF5 lazily (one handle per worker), and splits **validation** = **first num_val_items rows**, training = the rest (inference mode returns all rows). Finetuning uses `FineTuneLazyDataset`, which additionally joins the HDF5 rows with a label file (CSV/parquet keyed by RINPERSOON) and keeps only the intersection.

3 Model Architecture

The encoder (class `Transformer` in `transformer/transformer.py`) is a standard pre-norm/ReZero transformer stack, with two life2vec-specific twists: **Time2Vec temporal embeddings** and **Performer (FAVOR+) attention** with local attention heads.

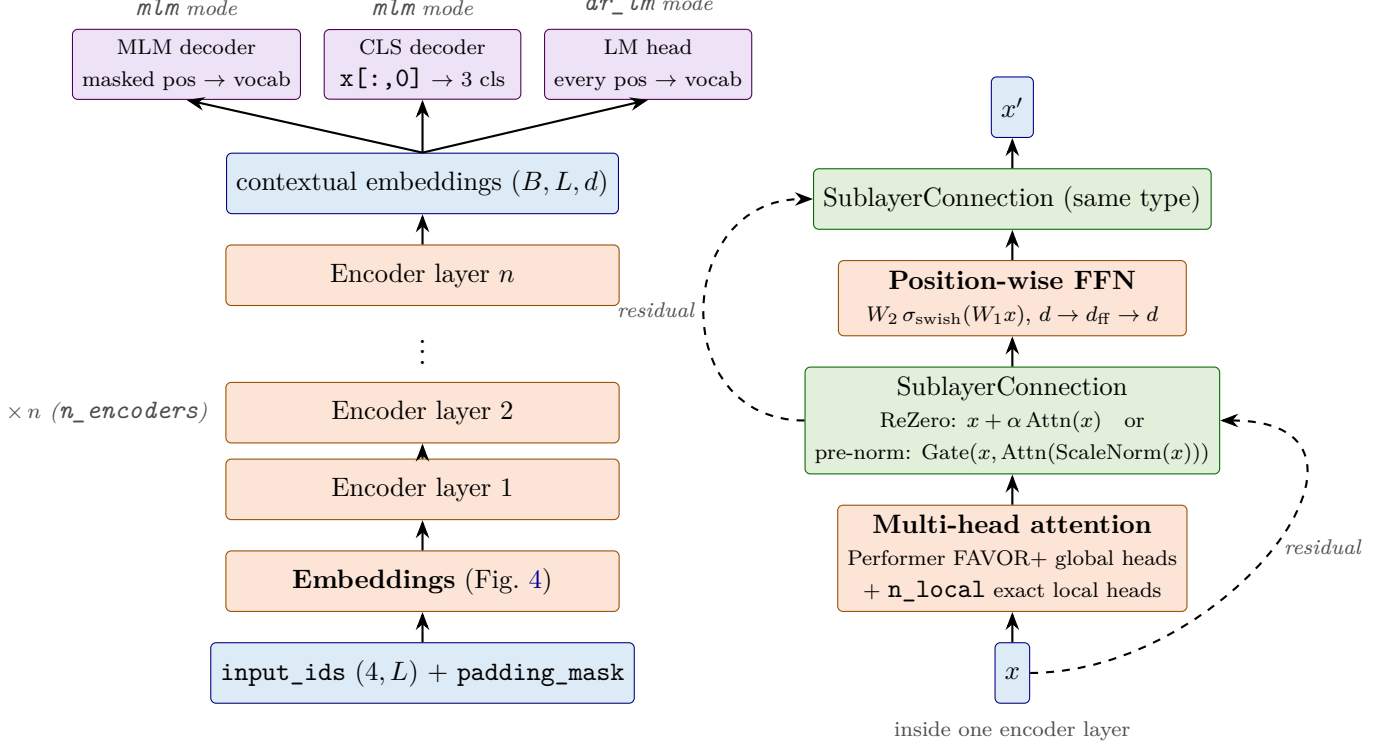


Figure 3: Left: the full model (`TransformerEncoder` in `transformer/models.py`). One shared encoder; the heads differ by pretraining mode. Right: one `EncoderLayer` (`transformer/transformer_utils.py`). Between layers, padded positions are re-zeroed via `einsum("bsh,bs->bsh", x, padding_mask)`.

3.1 Embedding block: how a token meets its time

Instead of BERT’s learned position embeddings, each token embedding is combined with **Time2Vec** encodings of *age* and *calendar time* (`abspos`), and a small *segment* embedding. Time2Vec, implemented in `transformer/embeddings.py`, maps a scalar τ to a d -dimensional vector

$$\text{t2v}(\tau) = [f(\tau w + b) \parallel \tau w_0 + b_0], \quad f = \cos \text{ (age)}, f = \sin \text{ (abspos)},$$

i.e. $d - 1$ periodic features plus one linear feature, with learned frequencies. Each additive term enters through its own **ReZero residual** $x \mapsto x + \alpha \cdot \text{t2v}$ with a learned scalar α initialized at 0 — so the model *starts as a pure bag-of-tokens* and learns how much time information to inject.

3.2 Attention: Performer with local heads

Full $O(L^2)$ softmax attention is *deprecated* in this codebase (`transformer/attention.py` raises if you ask for it). Instead, `attention_type: performer` uses the FAVOR+ approximation, implemented in `transformer/performer.py` as a wrapper around the `performer_pytorch` package:

$$\text{Attn}(Q, K, V) \approx \hat{D}^{-1}(\phi(Q)(\phi(K)^\top V)), \quad \phi = \text{positive random features (num_random_features)},$$

which is linear in sequence length. Of the `n_heads` heads, `n_local` are **exact sliding-window heads** (window `local_window_size`) that capture short-range structure (within-event and neighbouring-event patterns), and the remaining global heads use FAVOR+ for long-range structure. For the

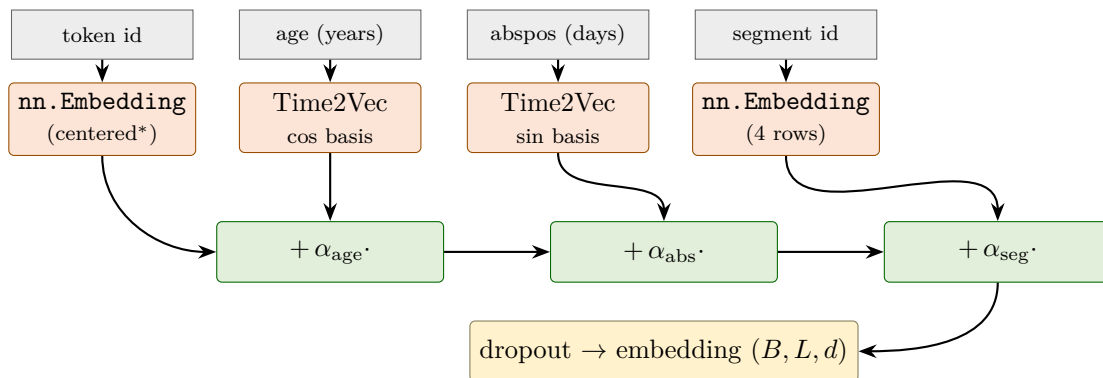


Figure 4: The embedding block (`Embeddings.forward`). ReZero scalars α start at 0. *If `parametrize_emb` is true, the token embedding matrix is mean-centered on every forward pass (ignoring PAD and placeholder ids), optionally ℓ_2 -normalized. The first 5 positions of each Time2Vec output are zeroed (`pos[:, :5] *= 0`) so the background prefix stays time-free.

GPT mode, the same module runs with `causal_attn: true` so each position can only attend to its past. Projection matrices are periodically redrawn during training (`redraw_projection_matrix`, called at each epoch end).

3.3 Heads

MLM decoder (`MaskedLanguageModel`). Gathers the hidden vectors at the masked positions (`target_pos`), applies $\text{dropout} \rightarrow \tanh(Vx) \rightarrow \ell_2\text{-norm} \rightarrow g \cdot W_{\text{out}}$, where W_{out} is **weight-tied** to the token embedding matrix (`weight_tying: wt`) and g is a learned temperature initialized to $\sqrt{d}$. Output: logits over the vocabulary for each masked slot.

CLS decoder (`CLS_DecoderS`). Takes the hidden state of position 0 (the [CLS] token), applies $\text{Linear} \rightarrow \text{swish} \rightarrow \text{ScaleNorm} \rightarrow \text{dropout} \rightarrow \text{Linear}$ to 3 classes (Sec. 4.1 explains the 3 classes).

AR LM head (`lm_head`). A single bias-free linear layer $d \rightarrow |V|$, weight-tied to the token embedding, applied to *every* position.

3.4 Reference hyperparameters

From `new_code/regular_hparams.txt`, the “small/base” config; the medium, medium2x and large files in the same folder scale these up:

<code>hidden_size</code> d	280	<code>n_encoders</code>	5
<code>hidden_ff</code> d_{ff}	2210	<code>n_heads</code>	10
<code>n_local</code> (exact heads)	7	<code>local_window_size</code>	38
<code>num_random_features</code>	436	<code>max_length</code>	512
<code>norm_type</code>	rezero	<code>hidden_act</code>	swish
<code>dropouts</code> (att/ff/dec/emb)	0.1	<code>weight_tying</code>	wt
<code>learning_rate</code>	5e-3 (OneCycle)	<code>weight_decay</code>	0.01

4 Step 1 — Pretraining (`pretrain.py`)

`pretrain.py` is a thin PyTorch-Lightning launcher: it reads the hparams file, builds train/val dataloaders over the HDF5, instantiates `TransformerEncoder`, and calls `Trainer.fit`. The one switch that changes everything is

`training_task: mlm` (default, BERT-style) vs. `training_task: ar_lm` (GPT-style).

4.1 BERT mode: masked LM + sequence-order CLS task

Masking is done **once, offline**, by `pipeline.py` when the HDF5 is created (not on the fly): `mlm_mask()` in `tasks/mlm.py` selects `mask_ratio = 30%` of the real tokens (never `[SEP]`/`[UNK]`, never position 0) and, of those selected: $\sim 80\%$ are replaced by `[MASK]`, $\sim 10\%$ are replaced by a random non-special token, and $\sim 10\%$ are left unchanged — BERT’s classic 80/10/10, with an optional “smart masking” variant that samples the random replacement from the *same attribute category*. An alternative masking: `event` mode masks whole events (sentences) instead of individual tokens.

In addition to MLM, `life2vec`’s auxiliary **sequence-order task** is kept: with probability 0.05 the event order of the document is *reversed* (label 1), with probability 0.05 it is *shuffled* (label 2), otherwise it is intact (label 0); the label is stored as `target_cls` and predicted from the `[CLS]` position. This forces the `[CLS]` summary to be sensitive to temporal coherence. Because the classes are imbalanced (90/5/5), the CLS loss uses inverse-frequency class weights (`cls_a` = $[1/0.9, 1/0.1, 1/0.1]$) and label smoothing 0.1.

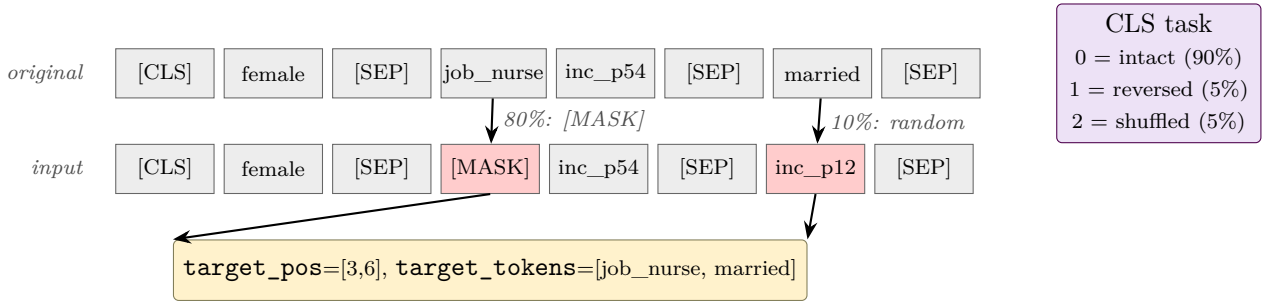


Figure 5: MLM encoding, done offline in `pipeline.py`. The model must recover the true tokens at `target_pos`; simultaneously the `[CLS]` head must detect whether the event order was tampered with.

The training loss (`TransformerEncoder.training_step`) is the `life2vec` weighted sum:

$$\mathcal{L} = 0.8 \underbrace{\text{CE}(\text{MLM logits}, \text{target_tokens})}_{\text{ignore_index}=0} + 0.2 \underbrace{\text{CE}_w(\text{CLS logits}, \text{target_cls})}_{\text{class-weighted, smoothed}}.$$

Logged metrics include top-5 macro accuracy/precision/recall/F1 over the vocabulary, CLS accuracy/F1, and perplexity $e^{\mathcal{L}_{\text{MLM}}}$.

4.2 GPT mode: autoregressive next-token prediction

For `training_task: ar_lm`, three things change:

1. **Data:** the HDF5 is read with `mlm_encoded=False` (the raw, unmasked token stream), and the collate function `collate_autoreg()` (`load_data.py`) builds the shifted (input, target) pairs on the fly.
2. **Attention:** `causal_attn: true` makes every head (FAVOR+ and local) causal.
3. **Head/loss:** the tied `lm_head` predicts the next token at every position; loss is cross-entropy with `ignore_index=0` (PAD), and perplexity is logged (clamped at e^{20} for safety).

`collate_autoreg` also implements a life-specific rule: each sequence is **truncated at the first death token** (inclusive) — there is nothing to predict after death.

4.3 Optimization and infrastructure (both modes)

- **AdamW** with no weight decay on biases, norms, and embedding-type parameters; **OneCycleLR** per step (1% linear warmup, div factors 10^4).
- Multi-GPU via Lightning **DDP** (`-devices, -ddpstrategy auto|ddp|ddp_mpi|gloo`); gradient clipping by norm; optional gradient accumulation and mixed precision (`training_precision`).

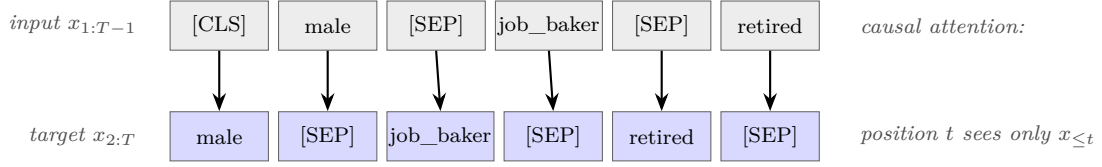


Figure 6: AR pretraining: teacher forcing with a one-token shift, on death-truncated sequences. The abspos/age/segment streams shift together with the token stream (the prefix keeps all 4 rows).

- **Checkpointing:** top-2 by `val_loss_track` plus an hourly time-based checkpoint; resuming supports both full-state resume and `load_model_weights_only`.
- Validation set = first `num_val_items` rows of the HDF5 (the pipeline shuffles people when building the file, so this is a random subset).

Run it:

```
python -m pop2vec.llm.src.new_code.pretrain -hparams hparams.txt -devices 4
```

5 Step 2a — Static Route: Embedding Inference (`infer_embedding.py`)

This script loads a pretraining checkpoint, **throws away the heads** (`model = model.transformer`), and runs every person’s sequence through the encoder once, producing a single d -dimensional vector per person. Two pooling options:

1. **Mean pooling (default).** Average the output token embeddings over real (non-padded) positions:

$$e_{\text{person}} = \frac{\sum_{t=1}^L m_t h_t}{\sum_{t=1}^L m_t}, \quad h_t = \text{encoder output at position } t, \quad m_t \in \{0, 1\}.$$

2. **Attention-weighted pooling (`use_attention_weighted_emb: true`).** The Performer never materializes an attention matrix, so `CustomSelfAttention._compute_approximate_attention()` reconstructs an *approximate* one from the random features of the **last** encoder layer, $A_{ij} \propto \phi(q_i)^\top \phi(k_j)$ (row-normalized). Token importance = *column* sums of A (how much attention each token *receives*), averaged over heads and normalized:

$$s_j = \frac{\sum_i A_{ij}}{\sum_{j'} \sum_i A_{ij'}}, \quad e_{\text{person}} = \sum_j s_j h_j.$$

This gives more weight to tokens the model itself deems salient. Implemented in `transformer/performer.py` (`get_attention_weighted_embedding`).

Embeddings are streamed to HDF5 batch by batch (`sequence_id`, `mean_emb`, optionally per-token embeddings with `save_token_embs`), then converted to **parquet** for easy joining with label files. A `needed_ids_path` parquet can restrict inference to a subset of persons. Config is a small JSON with `checkpoint_path`, `tokenized_path`, `emb_write_path`, `batch_size`.

6 Step 3a — Static Route: MLP Prediction (`train_simple.py`)

`train_simple.py` never touches the transformer: it consumes the *frozen* embedding parquet and a task file with labels, merges them on RINPERSON, and trains a **2-layer MLP** (SimpleMLP: `drop` $\rightarrow$ `Linear( $d, d$ )` $\rightarrow$ `LeakyReLU` $\rightarrow$ `drop` $\rightarrow$ `Linear( $d, k$ )`) per target column.

- **Task types** (declared per target in the config as `"target_column": {name: [type, num_outputs]}`): binary (BCE-with-logits, positive-class weight), categorical (weighted CE), numeric (MSE).

- **Class imbalance:** “Effective Number” weighting (`_en_weights`, Cui et al. style $w_c \propto (1 - \beta)/(1 - \beta^{n_c})$, clipped and normalized).
- **Early stopping** on a task-appropriate validation monitor (R^2 for numeric, best-threshold F1 for binary, macro-F1 for categorical), patience 10.
- **Threshold selection** (binary): after training, an exact sweep over all validation probabilities finds the F1-maximizing threshold (`_best_f1_threshold_torch`); it is stored *inside the model checkpoint* as buffer `best_thr` and reused at test time.
- **Couple mode:** if `PARTNER_KEY` is set, the partner’s embedding is concatenated (2d input) for couple-level outcomes.
- **Two run modes:** `train+val` (writes model + val metrics) and `test_only` (loads `load_model_path`, writes test metrics and per-person probability/label CSVs). The config integrity check makes the two modes mutually exclusive.

Results are appended to a CSV with the shared schema (`mode`, `task_file`, `model_name`, `target`, `type`, `model_path`, `val_acc ... val_r2`, `test_acc ... test_r2`, `LR`, `BATCH-SIZE`).

7 Step 2b — Finetuned Route (`finetune_runner.py` + `finetune_model.py`)

The finetuned route trains the *whole* network per task. `TransformerFT` (`finetune_model.py`) rebuilds the encoder from the pretraining hparams, loads the checkpoint weights (stripping the `transformer.` prefix; `pretrained_model_path`: `RANDOM` gives an untrained baseline), and attaches a small head:

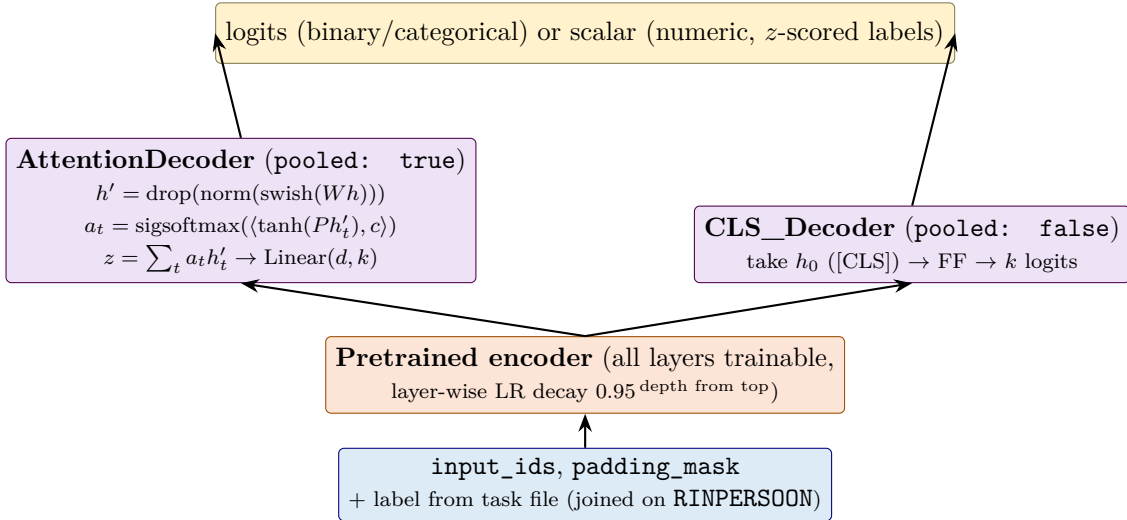


Figure 7: TransformerFT: pretrained encoder + lightweight task head. The default head (`pooled: true`) is a learned attention pooling over all token positions; the alternative uses the `[CLS]` vector only.

7.1 Training recipe (per target column)

`finetune_runner.py` loops over the targets in the config and, for each:

1. Builds `FineTuneLazyDataset` train/val loaders (labels from `train_path/val_path`; optional `balance_dataset` oversampling via `WeightedRandomSampler`; optional Effective-Number weighting via the `class-balanced-loss` flag). Numeric targets are z -scored with train-set μ, σ (stored in hparams and inverted at evaluation).
2. Optimizer: AdamW with **discriminative learning rates** — embeddings get $lr \cdot 0.95^n$, encoder layer i gets $lr \cdot 0.95^{n-i-1}$, the head gets $2lr$; OneCycle schedule; early stopping (patience 3) and checkpointing on `val_auc_epoch` (classification) or `val_r2_epoch` (numeric).
3. **Binary threshold:** at each validation epoch end, probabilities are all-gathered across GPUs and the exact best-F1 threshold is found; after training, the threshold from the

best checkpoint’s validation pass is written *into the checkpoint file* (`best_thr`), so test-time predictions are consistent.

4. In `test_only` mode, the saved checkpoint is loaded and `trainer.predict` runs over the test HDF5 rows; predictions are all-gathered, metrics computed against the test label file, and (optionally) per-person prediction CSVs written. A “safe dataset” wrapper replaces any out-of-vocabulary token id with PAD as a guard against vocab mismatches between checkpoint and data.

Results go to the *same* CSV schema as `train_simple.py` (Fig. 1), which is what makes static vs. finetuned comparisons one-liner joins.

8 Static vs. Finetuned, Side by Side

	Static (2a → 3a)		Finetuned (2b)
Scripts	<code>infer_embedding.py</code> <code>train_simple.py</code>	→	<code>finetune_runner.py</code>
Encoder weights	frozen (inference only)		all trainable (layer-wise LR decay)
Person representation	one pooled vector (mean or attention-weighted), computed once		attention-pooled or [CLS], recom- puted and adapted per task
Predictor	2-layer MLP on the vector		head on top of full encoder
Cost per new task	minutes (embeddings are cached)		full GPU training run per target
Class imbalance	Effective-Number loss weights		oversampling and/or EN weights
Binary threshold	best-F1 sweep on val, stored in <code>best_thr</code>		same mechanism, stored in Light- ning ckpt
Output	shared results CSV (+ prob/label CSVs)		shared results CSV (+ prediction CSVs)

Intuition for when each wins: the static route measures what the pretrained representation *already contains*; the finetuned route measures what the architecture can *extract when optimized for the task*. Running both on the same targets (identical CSV schema) is exactly how the codebase is set up to quantify that gap.

9 File Map and Config Cheat-Sheet

9.1 Who does what

File (under pop2vec/llm/src/)	Role
new_code/preprocess_data.py	clean raw tables; percentile-bin numerics; cap categoricals
new_code/create_life_seq_parquets.py	assemble per-person event sequences
new_code/pipeline.py	vocab creation; tokenize; offline MLM masking; write HDF5
tasks/mlm.py	<code>encode_document</code> : prefix, truncation, 4-stream tensor, 80/10/10 masking, CLS permutation task
new_code/load_data.py	datasets (CustomLazyHDF5Dataset, FineTuneLazyDataset); <code>collate_autoreg</code> (AR shift + death truncation)
new_code/pretrain.py	Step 1 launcher (Lightning)
transformer/models.py	<code>TransformerEncoder</code> : losses, metrics, optimizer for mlm / ar_lm
transformer/transformer.py	encoder stack; MLM/CLS heads; finetune forward variants
transformer/transformer_utils.py	<code>EncoderLayer</code> , ReZero/ScaleNorm/Gate, FFN
transformer/embeddings.py	token + Time2Vec(age, abspos) + segment embeddings
transformer/attention.py, performer.py	Performer FAVOR+ + local heads; approximate attention matrix; attention-weighted pooling
new_code/infer_embedding.py	Step 2a: person embeddings → HDF5/parquet
new_code/train_simple.py	Step 3a: MLP on frozen embeddings; results CSV
new_code/finetune_runner.py	Step 2b driver: per-target training/testing, thresholds, CSV
new_code/finetune_model.py	<code>TransformerFT</code> : encoder + head, layer-wise LR, metrics
new_code/generative_infer*.py	(GPT extras) sampling continuations from ar_lm models

9.2 Key config fields per step

Step	Required / notable keys
pretrain.py (hparams txt/yaml)	<code>checkpoint_dir</code> , <code>mlm_path</code> (the HDF5), <code>vocab_path</code> , <code>num_val_items</code> , <code>batch_size</code> , <code>epochs</code> ; <code>training_task</code> (mlm ar_lm), <code>causal_attn</code> (AR), <code>resume_from_checkpoint</code> , <code>load_model_weights_only</code>
infer_embedding.py (JSON)	<code>checkpoint_path</code> , <code>tokenized_path</code> , <code>emb_write_path</code> ; <code>batch_size</code> , <code>needed_ids_path</code> , <code>save_token_embs</code> , <code>use_attention_weighted_emb</code>
train_simple.py (JSON)	<code>emb_path</code> , <code>target_column</code> = {name: [type, k]}, <code>task_file</code> , <code>model_name</code> , <code>result_dir</code> ; train mode: <code>train_path</code> , <code>val_path</code> , <code>model_save_dir</code> ; test mode: <code>test_only</code> , <code>test_path</code> , <code>load_model_path</code> ; optional <code>PARTNER_KEY</code> (couple mode)
finetune_runner.py (JSON)	<code>sequence_encoded</code> (HDF5), <code>task_file</code> , <code>target_column</code> , <code>result_dir</code> ; train: <code>train_path</code> , <code>val_path</code> , <code>model_save_dir</code> , <code>pretrained_model_path</code> , <code>pretrained_model_hparams</code> , <code>model_name</code> ; test: <code>test_only</code> , <code>test_path</code> , <code>load_model_path</code> ; knobs: <code>pooled</code> , <code>balance_dataset</code> , <code>class-balanced-loss</code> , <code>layer_lr_decay</code> , <code>devices</code> , <code>save_predictions</code>

10 Frequently Confusing Points (Read This Twice)

1. **Masking is offline.** Unlike RoBERTa-style dynamic masking, the [MASK] pattern is fixed when `pipeline.py` writes the HDF5. The dataloader flag `mlm_encoded` simply says “this file contains the target columns”: it must be `True` for MLM pretraining and `False` for AR pretraining and for inference (which also flips on returning `sequence_id`).
2. **One encoder, two objectives.** `mlm` and `ar_lm` share `Transformer` exactly; the differences live in the batch format (pre-masked vs. shifted), the head (gathered MLM decoder vs. full-sequence tied LM head), and `causal_attn`. A checkpoint remembers its hparams, so `load_from_checkpoint` reconstructs the right variant automatically.

3. **input_ids** is $(B, 4, L)$, **not** (B, L) . Row 0 = token ids, row 1 = abspos (days), row 2 = age, row 3 = segment. Everything downstream (`x[:,0]`, `x[:,1]`, ... in `Transformer.forward`) depends on this order.
4. **The val split is positional.** Validation = the *first* `num_val_items` rows of the HDF5; randomness comes from the pipeline shuffling people at creation time. Keep `num_val_items` consistent between runs to compare losses.
5. **Attention is approximate everywhere.** Full softmax attention is deliberately disabled; both training and the “attention matrix” used for attention-weighted embeddings are Performer approximations. The extracted importance scores come from the *last* layer only.
6. **Two different train_simple.py exist** — `new_code/train_simple.py` (documented here) and an earlier `evaluation/prediction_settings/train_simple.py` from which `finetune_runner.py` imports `_en_weights`. They share the CSV schema and the MLP.
7. **Binary decisions are threshold-tuned, not 0.5.** Both routes sweep validation probabilities for the F1-optimal threshold and persist it in the model file (`best_thr`); test-time labels use that stored threshold.
8. **Numeric targets are z-scored during finetuning.** Predictions are un-normalized with the stored μ, σ before computing MAE/R^2 — if you ever see wildly wrong scales, check that these made it into the checkpoint.
9. **Death truncation only applies to AR.** MLM sequences keep everything within the time window; `collate_autoreg` cuts at the first `[death]_[death]` token because nothing after it should be generated.

Reference: Savcisens et al., “Using Sequences of Life-events to Predict Human Lives” (life2vec), arXiv:2306.03009. This codebase extends it with the AR (GPT) objective, Performer attention with local heads, attention-weighted static embeddings, and the dual static/finetuned evaluation harness on Dutch registry data.